BoardSketch

Build specification: how it works, how it is built, and how it is judged

Draw your circuit as a labeled diagram. BoardSketch asks the questions it cannot answer, builds a schematic, checks each part against its datasheet reference circuit, cites the page for every finding, and proposes fixes you approve.

Item	Decision
Users	First-time makers and students building their first board
Scope	6-part library, labeled-diagram input, 5 deterministic rules, one fix loop, schematic view
Not in scope	PCB routing, KiCad export, free-hand sketch recognition, parts beyond the library
Build window	24 hours, team of 4
Score target	About 77/100 (estimate; see Section 9)

Status of facts in this document. Design choices and the software architecture are decisions. Specific datasheet values, fab rules, and tool APIs are marked **[verify]** and must be checked against the current source before the demo. The judging rubric is assumed (default weights) because the MRU Hacks 2026 criteria were not confirmed.

1. The problem

Beginners and makers who build their first circuit tend to hit the same few failures. Each one is found late, after time or money is spent. Public forum and vendor sources describe these patterns repeatedly:

- A footprint in a library does not match the real part, and the wrong parts get ordered (a forum post describes 20 resistors that did not fit).
- A decoupling capacitor is placed far from the chip, or left out.
- A fabrication house rejects a file for a problem the design tool did not flag.
- A ground pour is missing on one layer, and wireless range suffers (the board works but badly).
- Datasheet layout guidance is skipped, even though it is in the document.

Sources: KiCad forum (forum.kicad.info, topics 30203 and 16253), DigiKey forum thread 70102, and the Circuit.rocks build report (forum.circuit.rocks/post272). These are community sources, not a survey. The Reddit threads were not reachable during research; confirm the pattern there before presenting it as common.

Why existing tools miss these: they check against generic rules and their own libraries, not against the datasheet of the part you chose, and they run after you finish drawing, not while you design.

2. The solution in one page

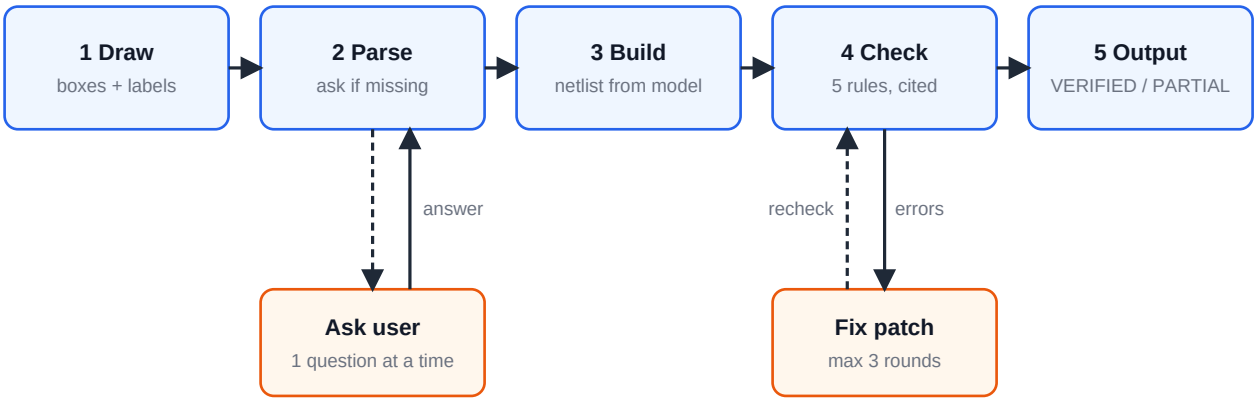
A user draws boxes and arrows with typed labels. The system does five things, in order:

Step	What happens	Who decides
1. Draw	User places parts as boxes and connects them with labeled arrows (for example GPIO2 to LED anode).	User
2. Parse and ask	The system reads the diagram. If a required value is missing (for example a sensor's supply voltage), it asks one question from a fixed list.	Rules (fixed question list)
3. Build	The model turns the diagram plus answers into a netlist (parts, nets, values). Output must match a schema.	Model, validated by schema
4. Check	Five rules run on the netlist. Each finding cites the datasheet page or rule it comes from.	Code (deterministic)
5. Fix and output	If errors remain, the model proposes a patch (max 3 rounds). The user approves each patch. Output is VERIFIED, PARTIAL, or NOT VERIFIED.	Code decides status; user approves fixes

Core rule of the product: the model proposes, the checks decide. A design is never shown as VERIFIED unless every rule that applies to it ran and passed.

Figure 1. Pipeline with the two loops

End-to-end flow with the two loops

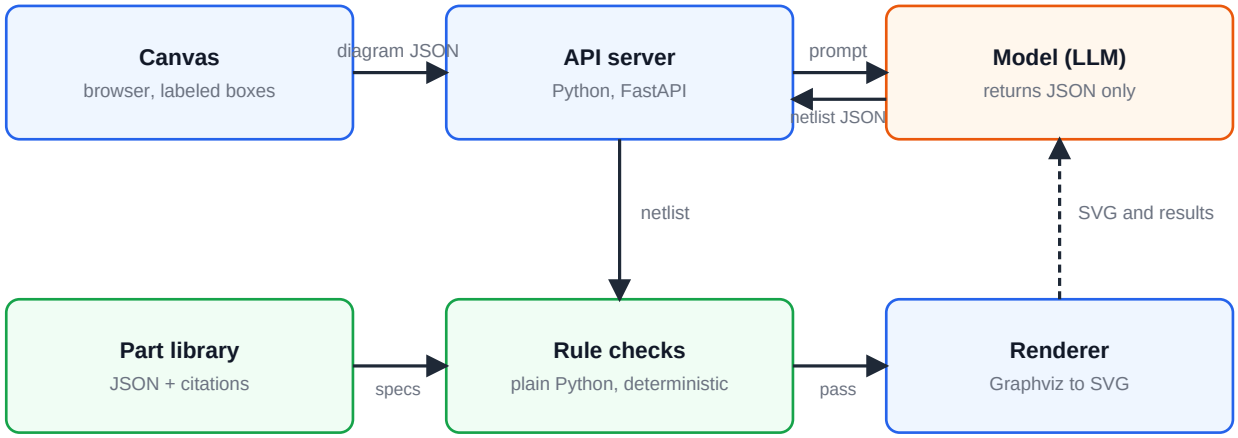


The fix loop repeats at most 3 times. The ask loop repeats only while questions remain, and is capped at 3 questions per session.

3. Architecture

Figure 2. Components and data flow

How the pieces talk to each other



The model never talks to the library or the checks directly. The API server is the only component that moves data between them, which keeps the pipeline testable without the model.

3.1 Components

Component	Responsibility	Tech (choice)	Owner
Canvas	Draw parts and labeled connections; export the diagram as JSON.	React + Vite. Use an existing whiteboard library if it exports labeled shapes [verify]. Otherwise a fixed-grid SVG editor.	Frontend
API server	Four endpoints (Section 4). Validates every input and output against schema.	Python 3 + FastAPI + Pydantic	Backend
Model	Parse answers, build netlist, propose fix patches. JSON output only.	A current Claude model via the API with structured (JSON) output [verify model ID and output mode in docs]	Model
Part library	Six parts, each with pins, supply ranges, required support, and citations.	JSON files in the repo	Backend
Rule checks	Five deterministic rules. Pure functions: netlist plus library in, findings out.	Python, no model calls	Backend
Renderer	Turn the netlist into an SVG schematic.	Graphviz dot to SVG [verify installed on demo machine]. Fallback: a plain list view.	Frontend
Demo cache	Stored outputs for the 3 demo drawings, used only when the live call fails, and labeled as cached.	JSON files on disk	Pitch/integration

4. How it works, exactly

4.1 Input: the diagram JSON

The canvas exports this structure. Pin names are typed by the user in the label, so the parser does not have to read free-hand symbols.

```
{
  "parts": [
    {"id": "U1", "type": "ESP32", "label": "ESP32"},
    {"id": "S1", "type": "TEMP_SENSOR", "label": "Sensor"},
    {"id": "D1", "type": "LED", "label": "LED"}
  ],
  "wires": [
    {"from": "U1.GPIO2", "to": "D1.ANODE"},
    {"from": "U1.GPIO4", "to": "S1.DATA"},
    {"from": "U1.3V3", "to": "S1.VCC"}
  ]
}
```

Rule: a wire end must name a part id and a pin that exists in that part's library entry. Anything else is rejected at Parse with a message that names the bad pin.

4.2 Parse and ask (no model needed)

- Each supported part has a list of **required parameters** in the library (for example *supply_voltage* for the sensor). If the diagram does not set one, the parser adds a question.
- Questions come from a **fixed template**, not from the model, so the wording is always correct. Example: "What supply voltage does the sensor run on? (3.3 V or 5 V)"
- Maximum 3 questions per session. The answer must be one of the listed options; free text is rejected and the question is repeated.
- The answers are saved on the part (for example *S1.supply_voltage = 3.3*) and shown in the final report, so the user can see every assumption.

4.3 Build: the model's job

The model receives: the diagram JSON, answers, and only the library entries for parts in the diagram (not the whole library). It must return a netlist that matches this shape:

```
{
  "components": [{"ref": "C1", "type": "CAP_100NF", "source": "ESP32 p.X"}],
  "nets": [{"name": "3V3", "pins": ["U1.3V3", "S1.VCC", "C1.1"]}, {"name": "GND", "pins":
["U1.GND", "S1.GND", "C1.2"]}],
  "values": {"S1.supply_voltage": 3.3}
}
```

- The model may add **only** components listed as required support in the library (for example a 100 nF decoupling capacitor). Any other added part is rejected.
- Every pin name must exist in the library. Every net must have at least two pins.
- If the output fails schema validation, the system retries once. If it fails again, the user sees "Could not build. Here is the error." No partial netlist is shown as a result.

4.4 Check: the five rules

Each rule runs on the netlist and the library. Every finding has the same shape: *rule, severity, part, message, citation*. If the library lacks the data a rule needs, the rule returns **UNVERIFIED**, never PASS.

ID	Rule	Pass condition (source)	Severity
R1	Every power pin is connected	Each pin tagged power in the library appears in a net [library]	Error
R2	Every ground pin is connected to GND	Each ground pin appears in the GND net [library]	Error
R3	Decoupling capacitor on each supply pin	A capacitor of the library's required value sits on each VDD pin's net. Value and placement rule come from the datasheet [verify page per part]	Error
R4	Net voltage is inside each pin's supply range	Net voltage (from values or the library) is within the pin's min and max [datasheet, verify]	Error
R5	GPIO drives an LED through a series resistor	LED anode or cathode path includes a resistor; GPIO current limit from the library is not exceeded [datasheet, verify]	Error

Important: the exact numbers (decoupling value, placement distance, current limit, voltage range) are not written in this document. They come from the datasheet of each part in the library and must be checked by hand before the demo.

4.5 Fix loop

- The model receives the netlist and the list of error findings only (not the whole history).
- It returns a **patch**, not a new netlist. Allowed operations: *add_component*, *add_wire*, *remove_wire*, *set_value*. Anything else is rejected.
- The patch is applied to a copy. The five rules run again on the copy.
- The user sees a diff (added and changed items) and approves or rejects it. Only approved patches are applied to the real netlist.
- Maximum 3 rounds. If errors remain, status is **PARTIAL** and the remaining errors are listed.

4.6 Output status

Status	When it is shown	Wording on screen
VERIFIED	All five rules ran and passed; no UNVERIFIED findings	Verified against the cited sources. Assumptions listed below.
PARTIAL	Errors remain after 3 rounds, or some rules are UNVERIFIED	Not verified. These findings remain: (list).
NOT VERIFIED	Build failed, or the library lacks data for a part	Could not verify. Reason: (message).

The word VERIFIED is only shown by code, never written by the model.

4.7 Worked example

Figure 3. The demo drawing and the checked result

Left: what you draw. Right: what the checked output contains.

Drawn: ESP32, sensor, LED, with GPIO2 driving the LED and GPIO4 to the sensor data line. The LED has no resistor drawn, and the sensor has no decoupling capacitor drawn. The checks flag both. The fix adds the decoupling capacitor (C1) and the series resistor (R1). Values shown are examples; the real ones come from the library.

Step	What the user sees
Parse	Question: "What supply voltage does the sensor run on? (3.3 V or 5 V)" User answers 3.3 V.
Build	Netlist with the drawn parts and no added parts yet.
Check	Findings: R3 error on the sensor's supply pin (no decoupling capacitor), with datasheet page. R5 error on the LED (no series resistor).
Fix	Patch: add C1 (100 nF example) on the sensor supply net; add R1 in series with the LED. User approves.
Recheck	All five rules pass. Status VERIFIED. Assumptions listed: sensor supply 3.3 V (user answer).

5. Model contract (what the model must always do)

- Output JSON only, matching the schema for the call (netlist or patch). No prose.
- Use only part ids and pin names from the provided library entries.
- Add only components that the library lists as required support.
- Never claim a rule passes. The checks decide.
- If unsure, return an empty patch with a reason field. The UI shows the reason.

Prompts and schemas are fixed text files in the repo, versioned, so a demo can be reproduced.

6. API (four endpoints)

Method and path	Input	Output
POST /api/parse	Diagram JSON	{ready: true} or {questions: [...]} (max 3)
POST /api/build	Diagram JSON + answers	{netlist, findings, status} or {error}
POST /api/fix	Netlist + error findings	{patch, netlist_after, findings_after, status}

Method and path	Input	Output
GET /api/library/{part}	Part id	Pins, required parameters, required support, citations

No database. Everything lives in memory during a session; demo state is a file on disk.

7. Technical execution plan

7.1 Scope lock (what makes the demo reliable)

- Library: exactly 6 parts (ESP32, one temperature sensor, LED, resistor, 100 nF capacitor, 3.3 V regulator). Each has hand-checked values.
- Rules: exactly 5 (R1 to R5). No more during the build.
- Demo inputs: exactly 3 diagrams, each tested end to end before the event.
- One live model call per demo run, so the AI part is real. Cached output is the fallback only.

7.2 Failure states (designed in, not patched later)

Failure	What the system does	What the user sees
Unknown part in diagram	Parse stops	"ESP32, sensor, LED, resistor, capacitor, regulator are supported. X is not."
Pin name not in library	Parse stops	Names the bad wire and the valid pins for that part
Model returns invalid JSON	Retry once, then stop	"Could not build. Here is the error."
Model adds unlisted part	Patch rejected	"Suggested part is not in the library; not applied."
Model API timeout or outage	Use cached output for the 3 demo drawings only	Label: "Cached result"
Fix loop does not converge	Stop after 3 rounds	Status PARTIAL with remaining errors
UNVERIFIED rule	Never shown as PASS	"Not checked: [reason]"

7.3 Build order (do not change)

- Hour 0-2: lock the diagram JSON and netlist schemas. Everything depends on these.
- Hour 2: model test. 5 labeled diagrams, count how many produce a valid netlist. Go/no-go gate.
- Hour 2-8: parts library (6 parts with citations) and the 5 rules, tested on hand-written netlists first (no model needed).
- Hour 6-12: connect the model to build, then the fix loop. Run on demo drawing 1 first.
- Hour 12-16: canvas, schematic render, approval UI. Sync from schematic back to drawing is cut unless everything else works.
- Hour 16: demo runs end to end, twice in a row. Feature freeze at 70 percent of build time.
- Hour 17-19: record backup video (by 80 percent). Fix only demo-path bugs.
- Hour 19-24: rehearse, pitch, rest. No new features after 80 percent.

7.4 Team split (4 people)

Role	Owns	Hands off to
Model and prompts	Build and fix prompts, schemas, go/no-go test	Backend (netlist format)
Backend	Library, 5 rules, API, validation, cache	Frontend (API contract)
Frontend	Canvas, schematic render, diff and approval view	Pitch (demo screens)
Pitch and integration	Datasheet verification, baseline timing, demo cache, backup video, pitch	Everyone (demo path)

Lock the API contract in hour 1, then the team works in parallel.

7.5 Stack choices to verify before building

- Whiteboard library: confirm it exports labeled shapes as data. [verify]
- Claude model ID and structured-output mode: check the current API docs. [verify]
- Graphviz installed on every demo machine, or the list-view fallback is used. [verify]
- Fab design rules are not used in the MVP. If added later, use the fab's published rules and cite them. [verify]

8. Demo and pitch (2 minutes 30 seconds)

Time	On screen	Say
0:00-0:15	Title slide, one problem statement	"Most first boards fail on a missing decoupling capacitor, and you find out after you have paid."
0:15-0:45	Draw the ESP32 + sensor + LED diagram	"I draw the circuit as I think about it. Labels are enough."
0:45-1:15	The question appears; user answers 3.3 V	"It asks what it cannot know, instead of guessing."
1:15-1:45	Findings appear with datasheet page; fix diff shows C1 and R1	"Every fix is tied to a page in the datasheet. You approve it."
1:45-2:15	VERIFIED badge, assumption list	"Here is what it checked and what it assumed. Nothing is hidden."
2:15-2:30	One-line pitch	"Draw it, and BoardSketch checks it against the datasheet before you order."

8.1 What to say about what is real

Be ready to answer directly: the parts library has 6 parts, the rules are 5, the demo uses 3 drawings that were tested. Show the cache label if the cached path was used. Do not describe the demo as general-purpose.

8.2 Measured impact number

Time one manual pass of the same 3 drawings (a check of parts, values and connections) and the tool on the same drawings. Put that number on the final slide. Do not use an estimate on stage.

8.3 Backup

Recorded run by hour 19, screenshots of each step, and cached output for the demo drawings.

9. Expected score (estimate, not judges' scores)

Criterion	Weight	Score (of 10)	Weighted	Why
Innovation	30%	7	21.0	Checks grounded in the datasheet's own reference circuit, with citations. Not a generic chatbot.
Technical execution	30%	8	24.0	Fixed scope, schema validation, designed failure states, cache fallback, one live call.
Impact	22%	7	15.4	One user group; one measured time number (must be measured before the event).
Presentation	18%	9	16.2	One drawing, one question, one caught error, one fix, one verified status.
Total			76.6 / 100	Assumes default rubric weights; event criteria not yet confirmed.

9.1 What lowers the score, and the fix

- A wrong pinout on stage: fixed by using only the 3 tested demo drawings.
- The model is unreliable: fixed by gate 1 (hour 2). If it fails, use a fixed template for the demo parts and say so.

- Scope creep: fixed by the feature freeze at hour 16.
- Impact number is an estimate: fixed by timing the baseline yourself.

10. Go/no-go gates

Gate	Test	If it fails
Hour 2	Model produces a valid netlist for 3 of 5 labeled diagrams	Use fixed templates for the 3 demo parts; keep the AI for the check and fix only
Hour 10	Checks and fix work on demo drawing 1	Show a labeled, scripted walkthrough; say it is scripted
Hour 16	Demo runs end to end twice in a row	Freeze features; rehearse; use cache for the failing step and label it

11. Next actions

#	Action	Owner	Time
1	Confirm the judging criteria and track on the official MRU Hacks 2026 page	Pitch	15 min
2	Lock the diagram JSON and netlist schemas; share the API contract	Model + Backend	1 h
3	Test model output on 5 labeled diagrams (gate 1)	Model	2 h
4	Write the 6-part library with datasheet citations and verified values	Backend + Pitch	2 h
5	Implement the 5 rules, tested on hand-written netlists	Backend	3 h
6	Connect model build and fix loop; run on demo drawing 1	Model + Backend	6 h
7	Canvas, schematic render, approval UI	Frontend	4 h
8	Time the manual baseline; lock 3 demo drawings and cache outputs	Pitch + Frontend	3 h